

Backend Fundamentals — Recap 2

POST, storing data, route params, and a complete CRUD API

Milestone: You built a complete CRUD API today.

1. POST — Sending Data to the Server

GET requests only ask for data — they have no real place to carry a chunk of data. POST requests carry a **body**: a package of data sent along with the request, separate from the URL. This is why a browser address bar can't test POST — typing a URL and hitting Enter only ever sends GET.

curl is a terminal tool that lets you send any HTTP method, including POST, with a body:

```
curl -X POST http://localhost:3000/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Buy milk"}'
```

Correction worth remembering: browsers *can* send POST (every login form does). It's specifically the address bar that only ever sends GET — not a browser limitation in general.

2. `express.json()` — Understanding the Body

Express does not automatically parse incoming JSON. Without this line, `req.body` is undefined, even if the client sent data correctly:

```
app.use(express.json());
```

Add it once, near the top, right after creating the app.

3. Storing Data (In-Memory)

A plain array (`let tasks = [];`) can hold data across multiple separate requests, because the array lives in the server's memory for as long as the process keeps running — it is not recreated per request. It resets to empty only when the server process itself restarts. This is a temporary stand-in for a real database, which comes later.

4. Route Parameters — Getting One Item by ID

A route like `/tasks/:id` uses `:id` as a placeholder. Visiting `/tasks/1` makes Express capture "1" (as text, not a number) and store it in `req.params.id`.

```
app.get('/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === Number(req.params.id));
  if (!task) {
    return res.status(404).send('Task not found');
  }
  res.send(task);
});
```

`Number(req.params.id)` converts the text id from the URL into a real number, so it can be compared correctly against numeric ids stored in the array.

5. Handling "Not Found" Properly

`.find()` returns undefined when nothing matches. Sending that straight back (`res.send(undefined)`) produces a blank page with a misleading **200 (success)** status — indistinguishable from something silently broken.

`if (!task)` reads as "if task is empty/missing." undefined is treated as falsy in JavaScript, so `!undefined` is true. Returning a real **404** status makes the failure explicit and honest instead of silent.

6. PUT and DELETE — Completing CRUD

```
app.put('/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === Number(req.params.id));
  if (!task) return res.status(404).send('Task not found');
  task.title = req.body.title;
  res.send(task);
});
```

```
app.delete('/tasks/:id', (req, res) => {
  const task = tasks.find(t => t.id === Number(req.params.id));
  if (!task) return res.status(404).send('Task not found');
  tasks = tasks.filter(t => t.id !== task.id);
  res.send(task);
});
```

`.filter()` builds a brand-new array containing only the items where the condition is true. `t.id !== task.id` keeps every task *except* the one matching the id to delete — that's how removal works on an array (you don't remove in place, you rebuild without it).

7. Real Bug Fixed: Variable Shadowing

The error `ReferenceError: Cannot access 'task' before initialization` came from a one-letter typo:

```
// Wrong – finding on the variable being declared, not the array
const task = task.find(t => t.id === Number(req.params.id));

// Correct – finding on the actual array
const task = tasks.find(t => t.id === Number(req.params.id));
```

Lesson to keep: when Node throws an error, it gives an exact file and line number (e.g. `index.js:30:18`) — check that line first before scanning the whole file.

8. Your Completed API

Action	Method	URL	Status on success	Status if missing
--------	--------	-----	-------------------	-------------------

Create	POST	/tasks	200	—
List all	GET	/tasks	200	—
Get one	GET	/tasks/:id	200	404
Update	PUT	/tasks/:id	200	404
Delete	DELETE	/tasks/:id	200	404

Note: right now POST still returns 200 instead of the more correct 201 (Created) — a real next improvement, not a mistake to fix urgently.

Next up: nodemon (auto-restart on save), then proper status codes, input validation, and eventually a real database instead of the in-memory array.